Windows GitHub and submission guide

The assignment requires a source repository with history, a report, saved evidence and a Jenkins pipeline. Student: Moneer. The local application is ready to run; these steps publish it and connect your own repository and Jenkins account. Commands below are for Windows PowerShell.

1 Verify the project before publishing

Open PowerShell in the extracted secure-workspace folder:

```
py -3.12 run.py setup
py -3.12 run.py test
py -3.12 run.py check
```

Expected: 40 passing tests, a successful full check, and all values in reports/gates.json equal to 0. Your actual output is authoritative; a new dependency advisory may block the gate later. The complete check requires internet access for the advisory lookup.

Read reports/tests.txt for the full-check test summary, reports/dependency-audit.txt for dependency findings and reports/gates.json for the gate decisions. Individual tests also save reports/tests.xml. The supplied earlier execution evidence is in evidence/final/ and comes from Linux. Preserve your own Windows outputs separately as described in step 3.

2 Create an empty GitHub repository

1. Sign in to GitHub.

2. Choose New repository and name it software-security-assignment.

3. Choose visibility according to your course's submission instructions. If private, ensure the examiner can access it.

4. Leave Add a README, .gitignore and license unselected. The project already contains files and history.

5. Create the repository and copy its HTTPS URL.

The ZIP already contains .git and the plan-before-code history. Do not run git init and do not upload only the files through GitHub's web page.

3 Save your Windows results and commit them

After a successful check, copy the results into a separate tracked evidence folder:

```
New-Item -ItemType Directory -Force evidence\windows | Out-Null
Copy-Item reports\* evidence\windows -Recurse -Force
git status
git log --oneline --decorate -6
```

Do this after your actual Windows run, so the folder is accurately labelled. The normal reports/ folder is ignored by Git to avoid committing every temporary run; evidence/windows/ is tracked.

Set your identity for future commits and save your own verified changes:

```
git config user.name "YOUR NAME"
git config user.email "YOUR GITHUB EMAIL"
git add evidence/windows
git commit -m "Record Windows verification results"
```

The earlier commits retain their original placeholder author and development order. You do not need to rewrite them. For later edits, review git status, then add the specific files you changed and commit them too.

4 Push the existing history

Replace the example URL with your repository URL:

```
git remote add origin https://github.com/YOUR-USERNAME/software-security-assignment.git
git push -u origin main
git push origin --tags
```

If a remote named origin already exists, inspect it with git remote -v. Use git remote set-url origin YOUR-REPOSITORY-URL if it points to a different repository. Do not force-push to an existing unrelated history.

Git may open a browser for authentication. After the push, refresh GitHub. Confirm that README.md, run.py, Jenkinsfile, helper/, tests/, docs/, evidence/ and submission/ are visible. Open the commit history and tags to confirm the initial plan and vulnerable-lab milestone are present.

5 Check a fresh clone

This checks that the repository works without relying on files from your original folder. In a different folder, use PowerShell:

```
git clone https://github.com/YOUR-USERNAME/software-security-assignment.git workspace-clone-check
cd workspace-clone-check
py -3.12 run.py setup
py -3.12 run.py test
py -3.12 run.py check
```

To see the running application, use py -3.12 run.py run, then open http://127.0.0.1:8000/docs. Use a second PowerShell window in that clone to run py -3.12 run.py demo.

6 Connect Jenkins to the repository

This is required to demonstrate the assignment's push and pull-request pipeline. If your course supplies Jenkins, use that server. Otherwise, set up a Jenkins instance and a Windows worker using the current Jenkins installation instructions. A full Jenkins server installation is separate from the Python application's setup.

The provided Jenkinsfile expects a worker labelled windows-python312. On that worker, the Jenkins account must be able to run git --version and py -3.12 --version. The worker needs network access to the advisory service; initial setup also needs internet to download the hash-approved packages.

In Jenkins:

1. Ensure Pipeline, Git, GitHub Branch Source, Credentials and JUnit capabilities/plugins are available.

2. Create a Multibranch Pipeline.

3. Add your GitHub repository as the branch source and configure credentials if required.

4. Enable branch discovery and pull-request discovery for the repository. Configure discovery to build both the intended pushed branches and PRs; do not leave a strategy that excludes the branch you are demonstrating.

5. Use the root Jenkinsfile as the pipeline definition. Assign the Windows worker its matching label.

6. Configure webhook delivery from GitHub to your reachable Jenkins endpoint, normally https://YOUR-JENKINS/github-webhook/, for push and pull-request events. Configure webhook validation on the server. A Jenkins address that only your PC can reach will not receive GitHub events without an appropriate reachable course/institution setup.

7. Save, scan the repository and inspect the first build.

Use a worker isolated from production credentials when running repository/PR code. Exact interface labels depend on your Jenkins and plugin versions. No hosted build is claimed until you perform these steps.

7 Demonstrate a push and a pull request

Make a small documentation change on main, commit it and push. Confirm Jenkins schedules a branch build. Then create a branch:

```
git switch -c verify-pipeline
```

Edit a sentence in README.md, then:

```
git add README.md
git commit -m "Verify pull request pipeline"
git push -u origin verify-pipeline
```

On GitHub, open a pull request from verify-pipeline into main. Confirm Jenkins schedules the PR build. The pipeline starts a fresh checkout, creates the virtual environment, checks dependencies, builds the helper, runs static and dependency analysis, starts temporary services and runs the HTTP tests. It archives results under Artifacts and publishes JUnit test results.

Save the actual branch build URL and PR build URL. If your server's configured Jenkins check appears on GitHub, add it as a required status check in the repository's branch protection/rules after confirming its exact name.

To demonstrate a blocking test gate, temporarily change a security assertion on a separate demonstration branch, push and inspect the failing build. Restore the assertion and push again. Keep the failure and corrected build links, and merge only the corrected version.

8 Finish the submission

1. Fill in your name, student number, repository URL and actual Jenkins build URLs on the report's final page. Keep the Word and PDF versions consistent after edits.

2. Submit the repository URL with its full history and tags. Ensure the examiner has access.

3. Submit the report in the required format and the saved test/analysis artifacts. The report is in submission/; tracked local evidence is in evidence/, and Jenkins also archives each run.

4. Complete the separate declaration in submission/ accurately. AI assisted implementation, testing, documentation and report drafting; describe your own subsequent review and changes as well.

5. Run the demonstration yourself and use docs/oral_exam_guide.md to practise explaining the boundaries and fixes.

Optional supply-chain experiments were not included. This guide is a checklist of actions to perform; it does not claim that your GitHub repository or hosted Jenkins runs already exist.

Official references

- Jenkins multibranch pipelines: https://www.jenkins.io/doc/book/pipeline/multibranch/
- GitHub webhooks: https://docs.github.com/en/webhooks/about-webhooks
- Jenkins installation: https://www.jenkins.io/doc/book/installing/